

Snake AI — Neuroevolution Project

Complete Implementation Guide

IBA Karachi · Introduction to Artificial Intelligence · Spring 2026
Instructor: Dr. Syed Ali Raza

Technique	Neural Networks + Genetic Algorithm (Neuroevolution)
Game	Snake (Python / Pygame)
Deliverables	Code, Report, Presentation with live demo
Deadline	6th May 2026

Table of Contents

- | | |
|-----|--|
| 1. | Project Overview & Research Questions |
| 2. | Architecture & How It All Works |
| 3. | Environment Setup |
| 4. | Phase 1 — Build the Snake Game |
| 5. | Phase 2 — Neural Network from Scratch |
| 6. | Phase 3 — Genetic Algorithm from Scratch |
| 7. | Phase 4 — Research Experiments |
| 8. | Phase 5 — GUI Overlay & Demo |
| 9. | Report Writing Guide |
| 10. | Research Papers to Read |
| 11. | Grading Checklist |
| 12. | Timeline |

1. Project Overview & Research Questions

You will build a Snake game in Python and train an AI agent to play it using neuroevolution — a technique that evolves a neural network's weights using a genetic algorithm, with no backpropagation and no ML library. The agent starts knowing nothing about the game and gradually learns to survive longer through selection pressure across many generations.

Core Research Questions

- Does a neuroevolved agent significantly outperform a random-action baseline?
- How does **population size** affect learning speed and final performance?
- How does **mutation rate** affect convergence — does higher mutation help or hurt?
- How does **network architecture** (number of hidden nodes) influence the agent?
- Does a NEAT-style evolving topology outperform your fixed-topology genetic algorithm?

■ The last question lets you compare your from-scratch implementation against a library-based advanced method (neat-python), which is explicitly allowed by the course rules.

What You Must Implement From Scratch (no ML library)

- The Snake game engine
- The feedforward neural network (forward pass only, no backprop)
- The genetic algorithm: fitness evaluation, selection, crossover, mutation

What Can Use a Library

- neat-python — for the advanced comparison baseline only
- pygame — for the game window
- numpy — for matrix operations in your NN
- matplotlib — for plotting fitness graphs

2. Architecture & How It All Works

The big picture: each 'agent' is a snake controlled by a neural network. The NN takes the current game state as input and outputs a direction. A population of agents plays simultaneously. After all die, the genetic algorithm selects the best ones, breeds new agents from them, and the cycle repeats.

Neural Network Inputs (11 values)

Input #	Description	Value
1	Danger straight ahead?	0 or 1
2	Danger to the right?	0 or 1
3	Danger to the left?	0 or 1
4	Currently moving left?	0 or 1
5	Currently moving right?	0 or 1
6	Currently moving up?	0 or 1
7	Currently moving down?	0 or 1
8	Food is to the left?	0 or 1
9	Food is to the right?	0 or 1
10	Food is above?	0 or 1
11	Food is below?	0 or 1

Neural Network Outputs (3 values)

- Output 0 → Go straight
- Output 1 → Turn right
- Output 2 → Turn left

The action with the highest activation is chosen (argmax).

Recommended Architecture

11 inputs → hidden layer (16 nodes, ReLU) → 3 outputs (softmax or argmax)

This gives you $11 \times 16 + 16 \times 3 = 224$ weights to evolve — small enough to train quickly, complex enough to learn real strategies.

Fitness Function

$\text{fitness} = \text{steps_survived} + (\text{food_eaten}^2 \times 100)$

Squaring food_eaten heavily rewards snakes that actually collect food, not just those that wander forever without dying. You can experiment with different formulas — this is a research decision.

3. Environment Setup

Prerequisites: Python 3.10+ installed. Use a virtual environment.

Install dependencies:

```
pip install pygame numpy matplotlib neat-python
```

Recommended project structure:

```
snake_ai/ ■■■ game.py # Snake game engine ■■■ neural_net.py # NN from scratch ■■■
genetic_alg.py # Genetic algorithm from scratch ■■■ agent.py # Connects NN + game ■■■
train.py # Main training loop ■■■ neat_compare.py # NEAT library comparison ■■■
visualize.py # Fitness plots ■■■ gui.py # Overlay for demo ■■■ config-neat.txt # NEAT
hyperparameters
```

4. Phase 1 — Build the Snake Game

Build the game first, completely separate from any AI. Make sure it works perfectly as a human-playable game before adding any AI.

game.py — Key class structure:

```
class SnakeGame:
    def __init__(self, w=480, h=480, render=True):
        self.w, self.h = w, h
        self.block = 20 # grid block size
        self.render = render # False = headless (fast training)
        self.reset()
    def reset(self):
        # Place snake in center, moving right
        # Place food randomly (not on snake)
        # Reset score, steps, done flag
    def get_state(self):
        # Returns numpy array of 11 binary inputs (see Section 2)
    def step(self, action):
        # action: 0=straight, 1=right, 2=left
        # Returns: reward, done, score
        # Reward: +10 food, -10 death, small -step penalty
    def _is_collision(self, pt=None):
        # Returns True if point hits wall or self
```

■ Critical

Add a step limit (e.g. $200 \times \text{snake_length}$) to prevent agents from surviving forever by just going in circles. This keeps training fast.

Key implementation details:

- Use a deque for the snake body — append head, pop tail on move
- Grid coordinates only (multiples of block size) — no pixel-level movement
- `render=False` mode lets you run training at full CPU speed with no window
- The `step()` method is the only interface the AI will use

Test it:

Add keyboard controls and play it yourself. Fix any bugs before moving to Phase 2.

5. Phase 2 — Neural Network from Scratch

This is the core AI component. You must implement it without sklearn, torch, or tensorflow.

neural_net.py — Complete structure:

```
import numpy as np
class NeuralNetwork:
    def __init__(self, layer_sizes): # layer_sizes e.g. [11, 16, 3]
        self.weights = []
        self.biases = []
        for i in range(len(layer_sizes) - 1):
            w = np.random.randn(layer_sizes[i], layer_sizes[i+1]) * 0.5
            b = np.zeros((1, layer_sizes[i+1]))
            self.weights.append(w)
            self.biases.append(b)
        def relu(self, x):
            return np.maximum(0, x)
        def forward(self, x): # x: numpy array shape (1, 11)
            for i, (w, b) in enumerate(zip(self.weights, self.biases)):
                x = x @ w + b
            if i < len(self.weights) - 1:
                x = self.relu(x)
            return x # raw logits, shape (1, 3)
        def predict(self, state):
            out = self.forward(np.array([state]))
            return int(np.argmax(out)) # 0, 1, or 2
        def get_weights_flat(self): # Returns all weights+biases as a 1D array (for the GA)
            return np.concatenate([w.flatten() for w in self.weights] + [b.flatten() for b in self.biases])
        def set_weights_flat(self, flat): # Rebuilds weights+biases from 1D array
            idx = 0
            for i, w in enumerate(self.weights):
                size = w.size
                self.weights[i] = flat[idx:idx+size].reshape(w.shape)
                idx += size
            for i, b in enumerate(self.biases):
                size = b.size
                self.biases[i] = flat[idx:idx+size].reshape(b.shape)
                idx += size
        def copy(self):
            nn = NeuralNetwork([1]) # dummy init
            nn.weights = copy.deepcopy(self.weights)
            nn.biases = copy.deepcopy(self.biases)
            return nn
```

■ Why flat weights?

The genetic algorithm treats the entire network as a single array of numbers (a 'genome'). `get_weights_flat()` and `set_weights_flat()` are the bridge between the NN and the GA.

6. Phase 3 — Genetic Algorithm from Scratch

The genetic algorithm is what drives learning. It mimics natural selection: better-performing agents are more likely to pass their 'genes' (weights) to the next generation.

genetic_alg.py — Full structure:

```
class GeneticAlgorithm:
    def __init__(self, pop_size=100, mutation_rate=0.1,
                 mutation_strength=0.3, elite_frac=0.1, layer_sizes=[11,16,3]):
        self.pop_size = pop_size
        self.mutation_rate = mutation_rate # % of genes mutated
        self.mutation_strength = mutation_strength # how much each gene shifts
        self.elite_count = max(1, int(pop_size * elite_frac))
        self.layer_sizes = layer_sizes
        self.population = [NeuralNetwork(layer_sizes) for _ in range(pop_size)]

    def evaluate_fitness(self, nn):
        # Run one full game episode with this NN, return fitness score
        game = SnakeGame(render=False)
        state = game.get_state()
        while True:
            action = nn.predict(state)
            _, done, score = game.step(action)
            state = game.get_state()
            if done:
                break
        steps = game.steps
        food = game.score
        return steps + food**2 * 100

    def select_parents(self, fitnesses):
        # Tournament selection: pick k random agents, return the best k = 5
        idx = np.random.choice(len(fitnesses), k, replace=False)
        return int(idx[np.argmax([fitnesses[i] for i in idx])])

    def crossover(self, parent_a, parent_b):
        # Uniform crossover: each gene randomly from parent A or B
        ga = parent_a.get_weights_flat()
        gb = parent_b.get_weights_flat()
        mask = np.random.rand(len(ga)) < 0.5
        child_genes = np.where(mask, ga, gb)
        child = NeuralNetwork(self.layer_sizes)
        child.set_weights_flat(child_genes)
        return child

    def mutate(self, nn):
        genes = nn.get_weights_flat()
        mask = np.random.rand(len(genes)) < self.mutation_rate
        genes[mask] += np.random.randn(mask.sum()) * self.mutation_strength
        nn.set_weights_flat(genes)
        return nn

    def evolve(self):
        # 1. Evaluate all agents
        fitnesses = [self.evaluate_fitness(nn) for nn in self.population]
        # 2. Elitism — keep top agents unchanged
        sorted_idx = np.argsort(fitnesses)[::-1]
        new_pop = [self.population[i].copy() for i in sorted_idx[:self.elite_count]]
        # 3. Fill rest with crossover + mutation
        while len(new_pop) < self.pop_size:
            p1 = self.select_parents(fitnesses)
            p2 = self.select_parents(fitnesses)
            child = self.crossover(self.population[p1], self.population[p2])
            child = self.mutate(child)
            new_pop.append(child)
        self.population = new_pop
        return max(fitnesses), np.mean(fitnesses), fitnesses[sorted_idx[0]]
```

Key Concepts Explained

Elitism

The top N agents always survive to the next generation unchanged. This prevents the algorithm from accidentally losing a really good solution through bad crossover. Start with elite_frac=0.1 (10%).

Tournament Selection

Instead of always picking the best agent (greedy), randomly pick 5 and take the best of those. This maintains population diversity — weaker agents occasionally get to reproduce.

Mutation Rate vs Mutation Strength

mutation_rate controls how many weights change; mutation_strength controls by how much. These are your main experimental variables — different values give very different learning curves.

7. Phase 4 — Research Experiments

This section is what separates a 10/15 project from a 15/15. Run systematic experiments and record results — this is the research component.

Experiment 1 — Baseline Comparison

- Run a random agent (random actions) for 100 episodes. Record mean survival steps and food eaten.
- Run your GA agent for 200 generations. Compare final performance to the random baseline.
- **Expected result:** GA agent survives 5–20× longer than random after training.

Experiment 2 — Population Size

- Run full training (200 generations) with `pop_size` ∈ {20, 50, 100, 200}
- Plot: best fitness per generation for each setting on the same graph
- Measure: generations needed to reach a threshold fitness (e.g. 500)
- **Expected result:** Larger populations converge more reliably but slower per generation.

Experiment 3 — Mutation Rate

- Fix `pop_size=100`. Vary `mutation_rate` ∈ {0.01, 0.05, 0.1, 0.2, 0.5}
- Plot: best fitness curves for each rate
- **Expected result:** Very low rates converge slowly; very high rates are noisy/unstable.

Experiment 4 — Network Architecture

- Test hidden layer sizes: [8], [16], [32], [16, 16] (two hidden layers)
- Fix all other hyperparameters. Compare convergence speed and peak fitness.

Experiment 5 — NEAT Comparison (Advanced Baseline)

- Train a NEAT agent using the `neat-python` library for the same number of generations
- Compare: final best fitness, generations to convergence, network complexity over time
- This is your 'advanced algorithm comparison' required by the course

■ For every experiment: run at least 3 independent trials and report mean ± std. Single-run results are not scientifically valid — random seeds affect outcomes.

What to Record Per Generation

```
# In your training loop, log this every generation: results = { "generation": gen,
"best_fitness": best, "mean_fitness": mean, "best_score": best_food_eaten,
"mean_score": mean_food_eaten, } # Save to CSV for later plotting
```

Plots to Include in Your Report

- Best fitness over generations (one line per experiment condition)
- Mean fitness over generations (shows population-wide learning)
- Bar chart: final performance across all experiments
- Screenshot of the snake playing at generation 1 vs generation 200

8. Phase 5 — GUI Overlay & Demo

The course requires a GUI. Since Snake is already visual, you just need an information overlay displayed on the game window during demo mode.

What to show on the overlay:

- Current generation number
- Best fitness score this generation
- All-time best fitness
- Current snake score (food eaten)
- A live mini fitness-over-generations graph (use `pygame.draw` or embed `matplotlib`)

Demo mode structure:

```
# demo.py – load best trained agent and run with render=True import pickle with
open('best_agent.pkl', 'rb') as f: best_nn = pickle.load(f) game =
SnakeGame(render=True, speed=15) # slower speed for presentation while True: state =
game.get_state() action = best_nn.predict(state) _, done, score = game.step(action)
draw_overlay(game.screen, generation=200, score=score) if done: game.reset()
```

■ Presentation tip

Record a short screen capture of generation 1 (random chaos) vs generation 100 (competent snake). Side by side = very compelling demo.

Training mode (fast):

```
# During training, run render=False for speed # Only render every N-th generation to
visually check progress RENDER_EVERY = 10 for gen in range(NUM_GENERATIONS): render =
(gen % RENDER_EVERY == 0) ga.evolve(render_best=render)
```

9. Report Writing Guide

Your final report must follow the AI_Report_Template format from the course. Here is how to fill each section for this project:

Abstract

2–3 sentences: what you did, what method, key result. E.g. 'We trained a Snake-playing agent using a genetic algorithm to evolve neural network weights. After 200 generations with a population of 100, the agent achieved a mean food score of X, compared to Y for a random baseline.'

Introduction

Motivate the problem. Why is game-playing AI interesting? What is neuroevolution? What gap does your work address? End with a list of your research questions.

Related Work

Summarize your 3 papers (1 paragraph each). Highlight what they did, what method they used, what result they found, and how your work differs or builds on them.

Methodology

Describe your NN architecture (diagram recommended), your genetic algorithm (pseudocode is fine), fitness function formula, and game state encoding. Be precise — someone should be able to re-implement from this section.

Experiments

List all 5 experiments. For each: what you varied, what you kept fixed, how many trials, and what you measured. Include your training hyperparameter table.

Results

Present all graphs and tables. For each result, explain what pattern you see. Don't just show graphs — interpret them.

Discussion

Why did you get these results? What surprised you? What limitations does your approach have? How does your GA compare to NEAT?

Conclusion

Summarize findings, answer each research question directly, suggest future work (e.g. bigger game grid, more complex architectures, applying to a different game).

References

All 3 papers in proper IEEE or APA format + any other sources.

10. Research Papers to Read

Paper 1 — The Foundational Algorithm (Must Read First)

Citation: Stanley, K. O., & Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation*, 10(2), 99–127.

What to read & why: The original NEAT paper. Read Sections 1–4 (introduction, background, the NEAT method, experiments). This is your primary theoretical reference. It explains: why evolving topology matters, how speciation prevents competing conventions, and how historical markings allow crossover between different structures. Available free at: [semanticscholar.org](https://www.semanticscholar.org/search?q=Stanley+Miikkulainen+2002+NEAT) (search 'Stanley Miikkulainen 2002 NEAT')

Where to cite: Use for: [Introduction](#), [Methodology](#), [Related Work](#)

Paper 2 — NEAT Applied to Game Playing

Citation: Efficient Application of Neuroevolution for Competitive Multiagent Learning. (2021). [arXiv:2105.10907](https://arxiv.org/abs/2105.10907)

What to read & why: Applies NEAT to a Pong environment — the closest published work to your project. Read how they define the observation space, fitness function, and how quickly NEAT converges compared to RL baselines. Available free at: arxiv.org/abs/2105.10907

Where to cite: Use for: [Related Work](#), [Experiments \(comparison baseline\)](#)

Paper 3 — Hyperparameter Sensitivity

Citation: Bailey, D. (2021). Adaptability of Improved NEAT in Variable Environments. [arXiv:2201.07977](https://arxiv.org/abs/2201.07977)

What to read & why: Investigates how improvements like population size and recurrent connections affect NEAT's performance in changing environments. Directly supports your Experiments 2–4 (population size, mutation, architecture). Available free at: arxiv.org/abs/2201.07977

Where to cite: Use for: [Related Work](#), [justifying your experimental design](#)

All three papers are freely available. Download PDFs from [arXiv](#) or [Semantic Scholar](#). Read each paper at least twice — once for the big picture, once for the technical details.

11. Grading Checklist

Use this before submission to verify everything is complete.

Code

- Snake game engine (game.py) works correctly as human-playable game
- Neural network implemented from scratch — no ML library used
- Genetic algorithm implemented from scratch (selection, crossover, mutation, elitism)
- Training loop runs and produces improving agents over generations
- NEAT comparison using neat-python library
- Results logged to CSV per generation
- Demo mode with GUI overlay showing generation/fitness/score
- Code is clean, commented, and runnable

Research / Report

- Clear problem statement and 4–5 research questions defined
- All 3 papers cited and summarized in Related Work
- Methodology section describes NN architecture and GA in enough detail to reproduce
- 5 experiments conducted with at least 3 independent trials each
- Fitness plots generated for all experiments
- Results section interprets (not just shows) all graphs
- Discussion compares GA vs NEAT and explains findings
- Conclusion answers each research question directly
- Report follows the AI_Report_Template format

Presentation

- Slides cover: Introduction, Methodology, Experiments, Results, Discussion, Future Work
- Live or recorded demo of agent playing Snake (generation 1 vs late generation)
- Side-by-side comparison: random agent vs trained agent
- NEAT comparison results shown
- Fitness curve animation or video included

12. Suggested Timeline (to May 6th Deadline)

Week	Dates	Tasks	Deliverable
------	-------	-------	-------------

1	May 2–4	Set up project structure. Build Snake game. Test human playability.	Gameplay working
2	May 5–6	Implement NeuralNetwork class. Test forward pass. Implement get_weights.	Neural network working
3	May 7–8	Implement GeneticAlgorithm. Run first training. Watch agent learn.	First trained agent
4	May 9–10	Run all 5 experiments. Log results to CSV. Generate all plots.	Experimental results
5	May 11–12	Run NEAT comparison. Build GUI overlay. Record demo.	Demo ready
6	May 13–14	Write full report. Prepare presentation slides. Final review.	Report + slides
—	May 15	Buffer day. Fix any remaining issues. Submit.	Final submission

■ ■ **Deadline**

Final submission is May 6th 2026 at midnight. All deliverables (code, report, presentation) must be submitted together.

Good luck! The most important thing is to start early — getting the game and NN working in Week 1 makes everything else much easier.